Implementing K- Means clustering using differnent disce metrics

In [26]:
```
import numpy as np
```

In [27]:
```
#Euclidean Distance
def euclidean_distance(a,b):
    return np.sqrt(np.sum((a-b)**2))
```

In [28]:
```
#Manhattan Distance
def manhattan_distance(a,b):
    return np.sum(np.abs(a-b))
```

In [29]:
```
def kmeans_custom(X, max_iters=10):
    centroids = np.array([[1,0,0],[0,1,1]])  #2 centroids
    for i in range (max_iters):
        clusters = [[] for i in range(2)]

        #Assign points to clusters
        for point in X:
            distances = []
            for centroid in centroids:
                distances.append(manhattan_distance(point, centroid))

            cluster_index = np.argmin(distances)
            clusters[cluster_index].append(point)

        # Update Centroid
        new_centroids = []
        for cluster in clusters:
            new_centroids.append(np.mean(cluster, axis=0))

        new_centroids = np.array(new_centroids)
        if np.allclose(centroids, new_centroids):
            break
        centroids = new_centroids
    return np.array(centroids), clusters
```

In [30]:
```
x = np.array([[1,2,3,],[0,1,2],[3,0,5],[4,1,3],[5,0,1]])
print(x)
```

```
[[1 2 3]
 [0 1 2]
 [3 0 5]
 [4 1 3]
 [5 0 1]]
```

In [31]:
```
centroids_x, clusters_x = kmeans_custom(x)
```

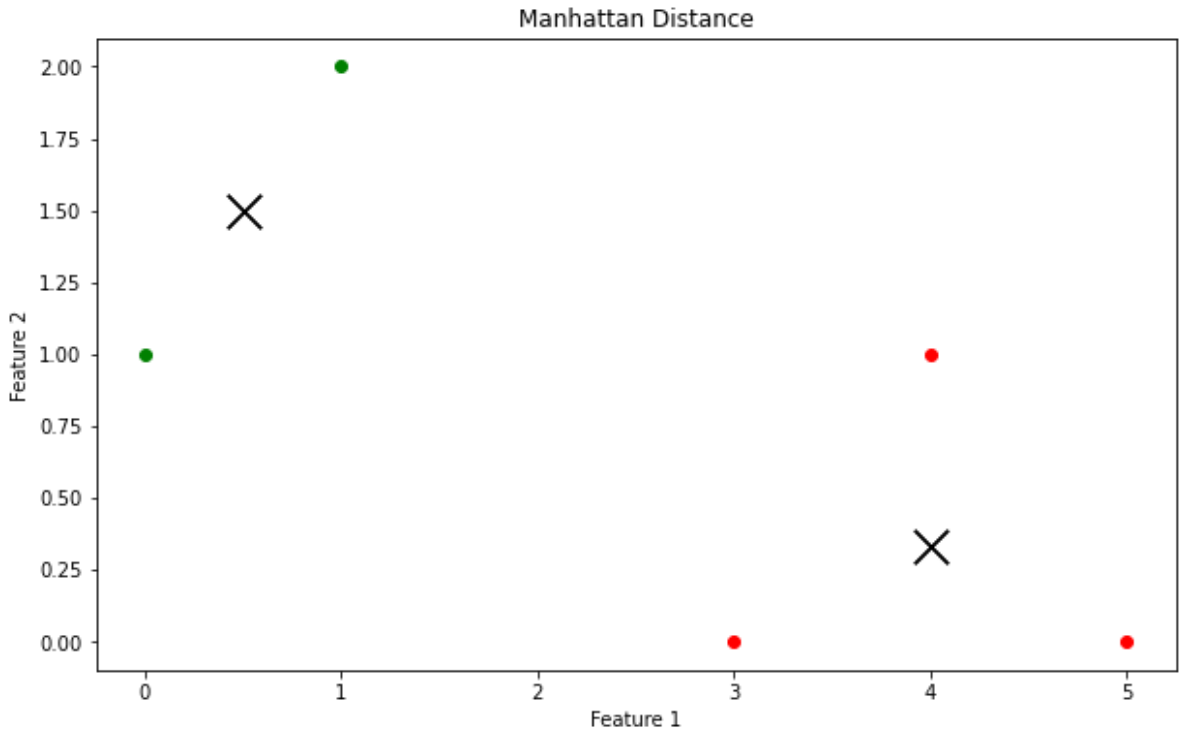
In [32]:
```
print("Centroids of X: \n", centroids_x)
print("Clusters of X: \n", clusters_x)
```

```
Centroids of X:
 [[4.         0.33333333 3.        ]
 [0.5        1.5        2.5       ]]
Clusters of X:
 [[array([3, 0, 5]), array([4, 1, 3]), array([5, 0, 1])], [array([1, 2, 3]),
array([0, 1, 2])]]
```

In [33]:
```
import matplotlib.pyplot as plt
def plot_clusters_2D(x,clusters,centroid):
    colors=['r','g']
    plt.figure(figsize=(10,6))
    for i, cluster in enumerate(clusters):
        cluster = np.array(cluster)
        plt.scatter(cluster[:,0], cluster[:,1], color = colors[i])
        plt.scatter(centroid[:,0],centroid[:,1], c="black", marker="x", s = 30
    plt.title("Manhattan Distance")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.show()
```

In [34]:
```
centroids_x, clusters_x = kmeans_custom(x) #Function call
```

In [35]:
```
plot_clusters_2D(x, clusters_x, centroids_x)
```



In [ ]: